



Case ranges

Document number:

P4040R0

Date:

2026-04-01

Audience:

SG22

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P4040/github

Source:

github.com/eisenwave/cpp-proposals/blob/master/src/case-ranges.cow



ABSTRACT: C2y added ranges in `case` labels, such as `case 0 ... 9:`. Such “case ranges” have also been supported as a C++ compiler extension for many years. This feature should be standardized for C++.

§ Contents

- 1 **Introduction**
- 2 **Motivation**
- 3 **Design**
 - 3.1 Design strategy
 - 3.2 Scoped enumerations
 - 3.3 Why not wait for pattern matching?
 - 3.4 What about pack expansion `case` labels?
- 4 **Implementation experience**
- 5 **Wording**
 - 5.1 [cpp.predefined]
 - 5.2 [stmt.label]
 - 5.3 [stmt.switch]



§ 1. Introduction

In 2024, [N3370] added support for `case` ranges to C2y. For example, the following two `switch` statements are equivalent:

C++26	C2y N3370
<pre>switch (next_char) { case '0': case '1': case '2': case '3': case '4': case '5': case '6': case '7': case '8': case '9': // ... }</pre>	<pre>switch (next_char) { case '0' ... '9': // ... }</pre>

The behavior here is obvious; `case '0' ... '9':` specifies ten cases in bulk.

This feature is already implemented in GCC and Clang, not just for C but also for C++. Since it is useful and widely supported, we should standardize existing practice and make it available to users in standard C++.

§ 2. Motivation

The benefit of the range syntax is obvious: when there are several contiguous cases, it is much more concise than listing each case individually.

While it would also be possible to handle case ranges using an `if` statement, this often requires splitting off some cases from the `switch`. This often results in a pattern like:

```
switch (c) {  
  case '+': consume_plus(); break;  
  case '=': consume_equals(); break;  
  default: {  
    if (c >= '0' && c <= '9')  
      consume_digit();  
  }  
}
```

There is an obvious asymmetry here, decreasing readability.

In addition to the feature being generally useful in C++, as mentioned above, it is an existing C2y feature. Providing it to users would make it easier to port C code to C++ and vice versa. Historically, C++ has always provided the control flow constructs of C, although `defer` may likely result in divergence.

§ 3. Design

§ 3.1. Design strategy

The overall strategy is to copy the semantics of the C2y feature as accurately as possible. Among other things, this includes design decisions such as:

- `case 0 ... 1ll` is permitted. That is, mixing types in case ranges is allowed. However, just like for existing `case` labels, this does not permit narrowing conversions (to the type of the condition).



EXAMPLE: A possible motivating example for mixing types in case ranges is:

```
case 0 ... std::numeric_limits<std::uint8_t>::max():
```

Especially in generic code, it may be annoying if the types of both sides of the *constant-range-expression* have to be of the same type.

- Overlapping ranges are disallowed. That is, the `switch` needs to be able to select one `case` label unambiguously.
- Empty case ranges such as `case 0 ... -1` are valid, but recommended to be diagnosed.
- Both ends are inclusive. That is, `case 0 ... 1` includes the values 0 and 1, and can be expressed as the range $[0, 1]$ in mathematical notation.

There is no obvious reason to deviate from the existing semantics of the C2y feature and of the C++ compiler extension; all these choices seem adequate.

A noteworthy quirk of the C2y feature is that a `case 0...9` is not valid because `0...9` is parsed as a *pp-number* from which no valid *token* can be formed, rather than two integers separated by an ellipsis. This problem cannot be fixed without altering the lexer, and having differences in lexing between C and C++ seems undesirable. It is recommended to always surround the `...` token with spaces, which works around the problem.



NOTE: This quirk is also documented at <https://gcc.gnu.org/onlinedocs/gcc/Case-Ranges.html>.

§ 3.2. Scoped enumerations

One feature exclusive to the C++ compiler extension is the support for scoped enumerations, such as in:



```
enum class Status {
    added,
    removed,
    error_general,
    error_invalid_argument,
};

void handle(Status s) {
    switch (s) {
        case Status::added ... Status::removed:
            break;
        case Status::error ... Status::error_invalid_argument:
            print_error(s);
            break;
    }
}
```

This part should also be standardized because it is useful. Many scoped enumerations organize enumerations into “blocks”, such as success statuses and error statuses, and case ranges allow for selecting such blocks.



NOTE: The range check does not undergo overload resolution for `operator<`, but takes place in terms of the underlying type of the enumeration.

§ 3.3. Why not wait for pattern matching?

Pattern matching provides very similar functionality:

P2688R4	C2y
<pre>next_char match { 'x' => f(); let c if (c >= '0' && c <= '9') => g(); }</pre>	<pre>switch (next_char) { case 'x': f(); break; case '0' ... '9': g(); break; }</pre>

Nonetheless, the case ranges are worth considering for C++29 for a variety of reasons:

- Case ranges are more concise. To be fair, a similar `'0' ... '9'` *match-case-pattern* could also be added to pattern matching, which closes that gap.
- `case` can be used in ways that pattern matching doesn't allow, like being nested inside other statements or with fallthrough into subsequent cases. Such uses are rare, but do exist.
- Case ranges can be used in code that is meant to compile in both C and C++, which is unlikely to ever be the case for pattern matching.
- Similarly, case ranges make it easier to port from C code that uses them to C++ code, and vice versa.
- Standardizing case ranges “legitimizes” existing C++ code that relies on the GNU extension for case ranges by giving it behavior specified by the standard. This increases portability

of old code bases that won't be modernized to use pattern matching.



§ 3.4. What about pack expansion **case** labels?

It may be worth considering a pack expansion **case** label, like:

```
template<auto... Args>
void f(int i) {
    switch (i) {
        case Args ...: break;
    }
}
```

However, this is not proposed, and is an entirely separate feature. The only thing it has in common with the proposed syntax is the “...”. Pack expansion **case** labels are also not strictly more general; this proposal offers **case** `0 ... 1'000'000'000:`, and doing the same via pack expansion would require a pack well past any compiler limits.

Adding case ranges also does not make it impossible to add pack expansion cases later; pack expansions use “...” as a unary suffix operator, not as a binary operator.



EXAMPLE: If both features existed, the following **case** would be disambiguated as an unexpanded pack on the left side of a *constant-range-expression*:

```
case Args... end:
```

This is fine because interpreting `Args...` as a pack expansion would make the construct as a whole invalid. Therefore, no change in meaning takes place, whether pack expansions are supported or not.

In conclusion, pack expansion **case** labels are not proposed, and case ranges do not prevent such a feature from being added in the future.

§ 4. Implementation experience

Case ranges were first implemented in GCC 2.0 (1992) and Clang 1.0 (2007), albeit as a GNU extension, not as a standard C2y feature. Both GCC and Clang currently provide the feature as proposed in both C and C++ mode.

MSVC does not support case ranges.

§ 5. Wording

The changes are relative to [\[N5032\]](#).

§ [cpp.predefined]



Add a feature-test macro to the table in [cpp.predefined] as follows:



```
__cpp_case_ranges 20XXXXL
```

§ [stmt.label]

Change [stmt.label] as follows:



A label can be added to a statement or used anywhere in a *compound-statement*.

label:

*attribute-specifier-seq*_{opt} *identifier* :

*attribute-specifier-seq*_{opt} *case* *constant-expression* :

*attribute-specifier-seq*_{opt} *case* *constant-range-expression* :

*attribute-specifier-seq*_{opt} *default* :

labeled-statement:

label *statement*

constant-range-expression:

constant-range-expression ... *constant-range-expression*

[...]

§ [stmt.switch]

Change [stmt.switch] paragraph 2 as follows:



2 If the *condition* is an *expression*, the value of the condition is the value of the *expression*; otherwise, it is the value of the decision variable. The value of the condition shall be of integral type, enumeration type, or class type. If of class type, the condition is contextually implicitly converted ([conv]) to an integral or enumeration type. If the (possibly converted) type is subject to integral promotions ([conv.prom]), the condition is converted to the promoted type.

3 Any statement within the *switch* statement can be labeled with one or more *case* *case* labels ~~as follows~~ of one of the forms:

*attribute-specifier-seq*_{opt} *case* *constant-expression* :

*attribute-specifier-seq*_{opt} *case* *constant-range-expression* :

where the *constant-expression* of the first form and each *constant-expression* of the *constant-range-expression* of the second form shall be a converted constant expression ([expr.const]) of the adjusted type of the ~~switch~~ *switch* condition. Let the value range of a case label *L* be:

- if L is of the first form, $[x, x]$, where x is the converted value of the *constant-expression*;
- otherwise, $[a, b]$, where a and b are the converted values of the first and second *constant-expression* of the *constant-range-expression*, respectively.

No two of the ~~case constants in the same switch~~ value ranges of labels associated with the same switch statement shall ~~have the same value after conversion~~ overlap.

Attach an example to the previous paragraph (now paragraph 3):



[Example:

```
unsigned int i = 0;
switch (i) {
    case -1:           // error: narrowing conversion from -1 to unsigned int
    case 0 ... 10ull:  // OK
    case 2 ... 3:      // error: value range overlaps with that of previous label
}
```

— end example]

Immediately following [stmt.switch] paragraph 2 (now split into two paragraphs), insert a new paragraph:



Recommended practice: Implementations should emit a warning when the value range of a case label is empty.



DRAFTING NOTE: This recommendation also exists in C2y.

Change [stmt.switch] paragraph 3 as follows:



There shall be at most one label of the form

attribute-specifier-seq_{opt} default :

~~within~~ associated with a switch statement.

Change [stmt.switch] paragraph 4 as follows:



Switch statements can be nested; a case or default label is ~~associated with~~ associated with the smallest ~~switch~~ switch statement enclosing it.

Change [stmt.switch] paragraph 5 as follows:



When the switch statement is executed, its condition is evaluated. ~~If one of the case constants has the same value as the condition, control is passed to the statement following the matched case-label. If no case constant matches the condition, and if~~

~~there is a default label, control passes to the statement labeled by the default label. If no case matches and if there is no default then none of the statements in the switch is executed.~~, and control may be passed to one of the statements labeled by a label associated with the switch statement, selected as follows:

- If the value of the condition lies in the value range of one of the case labels, control is passed to the statement labeled by that label. If the value is of enumeration type E, determining whether it lies in the value range takes place by first explicitly converting ([expr.type.conv]) the value range boundaries and the value of the condition to the underlying type of E.
- Otherwise, if there is a default label, control is passed to the statement labeled by that label.
- Otherwise, control is not passed to any of the statements in the switch statement.

§ 6. References

[N3370] Alex Celeste. Case range expressions, v3.1 (2024-10-01)

<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3370.htm>

[N5032] Thomas Köppe. Working Draft, Programming Languages — C++ (2025-12-15)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5032.pdf>